Pattern Recognition Lab Practice

Image Dataset Loading, Preprocessing, and Basic CNN Architecture

Course Area	Pattern Recognition Lab
Question Type	Python Coding Practice
Total Questions	6
Purpose	Student Practice for Lab Question 1

Instruction: Read each question carefully and write or understand the Python code. Each answer is followed by a short logic to help you understand the purpose of the code.

Question 1: Load, Resize, and Normalize an Image Dataset

You are given an image dataset for a binary classification task. Write Python code to load the dataset, resize the images into 128x128, and normalize the pixel values.

Answer

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Dataset path
dataset_path = "dataset"

# Normalize pixel values
datagen = ImageDataGenerator(rescale=1.0/255)

# Load dataset, resize images, and normalize pixel values
data = datagen.flow_from_directory(
    dataset_path,
    target_size=(128, 128),
    batch_size=32,
    class_mode='binary'
)
```

Logic: This code loads images from the dataset folder. `target_size=(128, 128)` makes all images the same size. `rescale=1.0/255` converts pixel values from 0 to 255 into 0 to 1.

Question 2: Load Dataset and Print Class Names

An image dataset is stored in separate folders, such as normal and disease. Write Python code to load the dataset and print the class names.

Answer

```
import tensorflow as tf

# Dataset path
dataset_path = "dataset"

# Load image dataset
```

```
data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    image_size=(128, 128),
    batch_size=32
)

# Print class names
print(data.class_names)
```

Logic: Folder names are automatically used as class names. This code helps check whether the dataset has been loaded correctly. `image_size=(128, 128)` also resizes the images during loading.

Question 3: Normalize Dataset Using Rescaling Layer

After loading an image dataset, write Python code to normalize the pixel values using a Keras Rescaling layer.

Answer

```
import tensorflow as tf
from tensorflow.keras import layers

# Dataset path
dataset_path = "dataset"

# Load dataset
data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    image_size=(128, 128),
    batch_size=32
)

# Create normalization layer
normalization_layer = layers.Rescaling(1.0/255)

# Apply normalization
normalized_data = data.map(lambda x, y: (normalization_layer(x), y))
```

Logic: Images usually have pixel values from 0 to 255. `Rescaling(1.0/255)` changes the values into 0 to 1. This is another simple way to normalize image data before using a CNN.

Question 4: Design a Basic CNN Architecture

Design a basic CNN architecture using TensorFlow/Keras. The model must include at least two Conv2D layers, MaxPooling layers, and a final Dense layer for binary classification.

Answer

```
from tensorflow.keras import layers, models
```

```
# Create CNN model
model = models.Sequential()

# First convolution and pooling layer
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)))
model.add(layers.MaxPooling2D((2, 2)))

# Second convolution and pooling layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Classification part
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

# Show model structure
model.summary()
```

Logic: Conv2D layers extract image features such as edges, shapes, and patterns. MaxPooling layers reduce the feature map size. The final sigmoid layer is suitable for binary classification.

Question 5: Write the Correct CNN Input Shape

Images are resized into 128x128 and they are RGB images. Write the first Conv2D layer with the correct input shape.

Answer

```
from tensorflow.keras import layers

# First CNN layer for RGB images
first_layer = layers.Conv2D(
    32,
    (3, 3),
    activation='relu',
    input_shape=(128, 128, 3)
)
```

Logic: The input shape is `(128, 128, 3)` for RGB images. The first two values represent image height and width. The last value 3 means Red, Green, and Blue channels.

Question 6: Create CNN Model with Normalization Inside the Model

Design a CNN model where the image normalization is included as the first layer of the model.

Answer

```
from tensorflow.keras import layers, models
```

```
# Create CNN model
model = models.Sequential()

# Normalize input image pixels
model.add(layers.Rescaling(1.0/255, input_shape=(128, 128, 3)))

# CNN feature extraction layers
model.add(layers.Conv2D(32, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Classification layers
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.summary()
```

Logic: The Rescaling layer normalizes pixels inside the CNN model. After normalization, Conv2D and MaxPooling layers extract image features. The final Dense layer with sigmoid gives binary classification output.

Short Notes for Students

Term	Meaning
flow_from_directory()	Loads images from folder based dataset.
image_dataset_from_directory()	Loads images from folders and can resize them directly.
target_size or image_size	Makes all images the same size.
rescale or Rescaling	Normalizes pixel values from 0 to 255 into 0 to 1.
Conv2D	Extracts image features.
MaxPooling2D	Reduces feature map size.
Flatten	Converts 2D feature maps into a 1D vector.
Dense	Performs final classification.
sigmoid	Used for binary classification.